

The Design and Formal Analysis of the Hierarchical Key Distribution System

John G. Underhill

Abstract

The Hierarchical Key Distribution System (HKDS) is a symmetric key establishment mechanism that derives operational transaction keys from two server-held root secrets and per-device identifiers using SHAKE and KMAC. HKDS provides token-based key diversification followed by a per-transaction cache of one-time encryption and authentication keys. This paper gives a formal security analysis of HKDS in a game-based framework. It formalizes confidentiality, message authenticity, token authenticity, forward secrecy under erasure, post-compromise security, partial server-root compromise, and multi-device separation.

The analysis shows that authenticated HKDS provides IND-CPA confidentiality for single-use 128-bit transaction-key blocks and UF-CMA authenticity for fixed 128-bit KMAC tags, subject to the PRF security of SHAKE and KMAC and correct counter discipline. Token forgery reduces to KMAC security under the embedded device key plus the generic probability of guessing a 128-bit tag. Forward secrecy and post-compromise security follow from erasure of expired token and cache material and from the independence of the server token key from the embedded device key derivation path. The bounds distinguish the 128/256/512-bit root-key profiles from the fixed 128-bit transaction-key and authentication-tag widths used by the reference implementation.

Contents

1	Introduction	3
1.1	Related Work	3
2	Engineering Description of HKDS	4
2.1	Roles and Principal Values	4
2.2	Token Exchange	4
2.3	Transaction-Key Cache and Message Protection	5
2.4	Reference Algorithms	5
3	Preliminaries and Assumptions	7
3.1	Notation	7
3.2	Primitive Assumptions	8
4	Security Model	8
4.1	Adversary	8
4.2	Oracles	9
4.3	Security Notions	9
5	Structural Properties	9
5.1	Domain Separation	9
5.2	Deterministic Recoverability	10
6	Security Results	10
6.1	Message Confidentiality	10
6.2	Message Authenticity	11
6.3	Token Authenticity	11
6.4	Forward Secrecy Under Erasure	11
6.5	Post-Compromise Security	12
6.6	Multi-Device Separation	12
6.7	Partial Server-Root Compromise	12
7	Concrete Bounds	12
8	Implementation and System Considerations	13
8.1	Erasure and Memory Hygiene	13
8.2	Counter Discipline and Replay	13
8.3	Authenticated and Unauthenticated Modes	14
8.4	Parallel APIs	14
8.5	Side Channels and Reduced Rounds	14
9	Conclusion	14

1 Introduction

Distributed symmetric key systems are used in payment networks, point-of-sale terminals, automated teller machines, industrial devices, and other embedded environments where many clients must authenticate and protect short messages sent to a centralized transaction authority. The dominant historical construction in the payment domain is Derived Unique Key Per Transaction (DUKPT), standardized in the ANSI X9.24 family of standards. DUKPT derives transaction-specific keys from a device-specific base key and counter state. It has been operationally successful, but its sequential derivation structure and server-side work growth do not provide an ideal foundation for higher security profiles, high-volume stateless reconstruction, or post-compromise recovery.

HKDS replaces the DUKPT-style key progression with a two-root symmetric hierarchy. The server holds a master derivation key containing a base derivation key, a secret token key, and a key identifier. Each client is provisioned with an embedded device key derived from the base derivation key and the device identity block. A server-issued token, derived from the secret token key and an epoch-specific customization string, is encrypted and authenticated under values derived from the embedded device key. The client combines the recovered token with the embedded device key to expand a transaction key cache. In authenticated mode, each protected message consumes two consecutive cache entries, one for XOR encryption and one for KMAC authentication.

The design objective is to obtain deterministic server reconstruction without long-term per-device server state. The server can reconstruct any required transaction-key window from the received key serial number and the master derivation key. The client consumes and erases transaction keys sequentially. This creates a compact symmetric protocol whose principal security claims are confidentiality of protected messages, authenticity of authenticated ciphertexts, authenticity of token delivery, forward secrecy for erased transaction keys, and predictive resistance after client compromise followed by a fresh server token.

This paper formalizes these properties. Section 2 describes the protocol and the reference algorithms. Section 3 fixes parameters, notation, and assumptions. Section 4 defines the adversary and oracle model. Section 5 proves structural properties, including domain separation and deterministic reconstruction. Section 6 gives the security results. Section 7 states concrete bounds. Section 8 discusses implementation and system considerations. Section 9 concludes.

1.1 Related Work

HKDS is most closely related to deployed symmetric transaction-key systems such as DUKPT, specified in ANSI X9.24-3. The central engineering difference is that HKDS replaces a long sequential key derivation path with token epochs derived from a server-only token root and a per-device embedded key. This permits bounded reconstruction of transaction keys and provides an explicit post-compromise recovery mechanism when a fresh token is obtained from the server.

The proof style follows standard game-based cryptographic analysis, including sequence-of-games reasoning and oracle-based definitions for confidentiality and authenticity. The analysis also uses the standard principle that a PRF under a fresh and independent key gives a secure MAC up to the generic tag-guessing probability. HKDS uses SHAKE as an extendable-output PRF over fixed-width absorbed domains, while KMAC is used for token and message authentication. This differs from extract-and-expand constructions such as HKDF, but the security reasoning is analogous: independent domain inputs and keys are required for independent derived outputs.

2 Engineering Description of HKDS

2.1 Roles and Principal Values

An HKDS deployment contains a transaction server and many terminal clients. The server holds a master derivation key

$$\text{MDK} = (\text{BDK}, \text{STK}, \text{KID}),$$

where **BDK** is the base derivation key, **STK** is the secret token key, and **KID** is a 4 byte key identifier. The byte length of **BDK** and **STK** is selected by the HKDS profile: 16 bytes for HKDS-128, 32 bytes for HKDS-256, and 64 bytes for HKDS-512.

Each client has a 12 byte device identity block **DID**. In the reference implementation, the first four bytes of this block are the master-key identifier, and the remaining eight bytes form the terminal identity sub-block containing protocol flags, device mode, manufacturer identifier, and device identifier. The key serial number is

$$\text{KSN} = \text{DID} \parallel \text{KTC},$$

where **KTC** is a 32-bit big-endian transaction counter. The embedded device key is

$$\text{EDK} = \text{SHAKE}(\text{DID} \parallel \text{BDK}).$$

The client is provisioned with **EDK** and **DID**; it does not need to possess **BDK**.

The token-request counter is derived from the transaction counter by division by the cache size:

$$\text{TRC} = \left\lfloor \frac{\text{KTC}}{\text{HKDS_CACHE_SIZE}} \right\rfloor.$$

The custom token string and token MAC string are

$$\text{CTOK} = \text{TRC} \parallel \text{FN} \parallel \text{DID}, \quad \text{TMS} = \text{KSN} \parallel \text{MACName}.$$

The formal name **FN** is the seven byte ASCII string identifying the HKDS security profile, for example **HKDS128**, **HKDS256**, or **HKDS512**. The MAC-name string is the internal HKDS MAC label, such as **uKwe128**, **uKwe256**, or **uKwe512**. It is used as part of the token MAC customization string and is not the literal NIST algorithm name.

2.2 Token Exchange

The server derives an epoch token and an encrypted token body by

$$\text{TOK} = \text{SHAKE}(\text{CTOK} \parallel \text{STK}), \quad \text{Kpad} = \text{SHAKE}(\text{CTOK} \parallel \text{EDK}), \quad \text{ETOK} = \text{TOK} \oplus \text{Kpad}.$$

It then computes a fixed 16 byte token tag

$$\text{TAG}_t = \text{KMAC}_{\text{EDK}}(\text{ETOK}; \text{custom} = \text{TMS}).$$

The server returns $\text{ETOK} \parallel \text{TAG}_t$ to the client. The client recomputes **TMS**, verifies the tag under **EDK**, recomputes $\text{Kpad} = \text{SHAKE}(\text{CTOK} \parallel \text{EDK})$, and recovers $\text{TOK} = \text{ETOK} \oplus \text{Kpad}$ only if verification succeeds.

2.3 Transaction-Key Cache and Message Protection

After token recovery, the client derives

$$\text{TKC} = \text{SHAKE}(\text{TOK} \parallel \text{EDK}),$$

partitioned into 16 byte entries. With the default cache multiplier of four, the cache contains 42 entries for HKDS-128, 34 entries for HKDS-256, and 18 entries for HKDS-512. These values follow directly from

$$\text{HKDS_CACHE_SIZE} = \frac{\text{HKDS_CACHE_MULTIPLIER} \cdot \text{HKDS_PRF_RATE}}{\text{HKDS_MESSAGE_SIZE}},$$

using SHAKE rates 168, 136, and 72 bytes.

In unauthenticated mode, a client message M of 16 bytes is protected by one cache entry:

$$C = M \oplus K_i.$$

In authenticated mode, the client consumes two consecutive cache entries:

$$C = M \oplus K_{\text{enc}}, \quad \text{TAG} = \text{KMAC}_{K_{\text{mac}}}(C; \text{custom} = A),$$

where A is associated data supplied to the KMAC customization parameter. The server reconstructs the same key window from the received KSN, verifies the tag first, and decrypts only when verification succeeds.

2.4 Reference Algorithms

The following pseudo-code records the functional behavior relevant to the proof. It uses the same byte-array interpretation as the C implementation, but abstracts away routine pointer checks and error propagation.

Algorithm 1 Client state initialization

Require: Client state `state`, embedded key `edk`, device identity block `did`

- 1: copy `edk[0..HKDS_EDK_SIZE-1]` into `state.edk`
 - 2: copy `did[0..HKDS_DID_SIZE-1]` into `state.ksn[0..HKDS_DID_SIZE-1]`
 - 3: set `state.ksn[HKDS_DID_SIZE..HKDS_KSN_SIZE-1]` to zero
 - 4: set every entry of `state.tkc` to zero
 - 5: `state.cache_empty` $\leftarrow$ `true`
-

Algorithm 2 Client transaction-key extraction

Require: Client state `state`, output `tkey`

- 1: $c \leftarrow \text{BE8TO32}(\text{state.ksn}[\text{HKDS_DID_SIZE}..])$
 - 2: $j \leftarrow c \bmod \text{HKDS_CACHE_SIZE}$
 - 3: copy `state.tkc[j]` into `tkey`
 - 4: securely erase `state.tkc[j]`
 - 5: increment the big-endian 32-bit counter in `state.ksn`
 - 6: **if** $j = \text{HKDS_CACHE_SIZE} - 1$ **then**
 - 7: `state.cache_empty` $\leftarrow$ `true`
 - 8: **end if**
-

Algorithm 3 Client token decryption

Require: Client state **state**, encrypted token response **etok**, output token buffer **token**

Ensure: true on token acceptance, otherwise false

```
1: CTOK  $\leftarrow$  CLIENTGETCTOK(state)
2: TMS  $\leftarrow$  CLIENTGETTMS(state)
3: code  $\leftarrow$  KMACstate.edk(etok[0..HKDS_STK_SIZE-1]; TMS)
4: if code  $\neq$  etok[HKDS_STK_SIZE..] then
5:   return false
6: end if
7: token  $\leftarrow$  SHAKE(CTOK || state.edk)
8: for  $i = 0$  to HKDS_STK_SIZE-1 do
9:   token[i]  $\leftarrow$  etok[i]  $\oplus$  token[i]
10: end for
11: return true
```

Algorithm 4 Client cache generation

Require: Client state **state**, accepted token **token**

```
1:  $S \leftarrow$  SHAKE(token || state.edk) with output length HKDS_CACHE_SIZE * HKDS_MESSAGE_SIZE
2: for  $i = 0$  to HKDS_CACHE_SIZE-1 do
3:   copy  $S[i \cdot 16..(i+1) \cdot 16 - 1]$  into state.tkc[i]
4: end for
5: state.cache_empty  $\leftarrow$  false
```

Algorithm 5 Authenticated client encryption

Require: Client state **state**, plaintext **M**, associated data **A**

Ensure: Ciphertext and tag, or failure

```
1: if fewer than two cache entries remain then
2:   return failure
3: end if
4: extract next cache entry as  $K_{\text{enc}}$ 
5:  $C \leftarrow M \oplus K_{\text{enc}}$ 
6: extract next cache entry as  $K_{\text{mac}}$ 
7:  $T \leftarrow$  KMAC $K_{\text{mac}}$ ( $C$ ; custom =  $A$ ) truncated to 16 bytes
8: erase local copies of  $K_{\text{enc}}$  and  $K_{\text{mac}}$ 
9: return ( $C, T, \text{KSN}$ )
```

Algorithm 6 Server transaction-key reconstruction

Require: Server state `state`, output length `tkeylen`

- 1: $j \leftarrow \text{BE8TO32}(\text{state.ksn}[\text{HKDS_DID_SIZE}..]) \bmod \text{HKDS_CACHE_SIZE}$
 - 2: $\text{DID} \leftarrow \text{state.ksn}[0..\text{HKDS_DID_SIZE}-1]$
 - 3: $\text{EDK} \leftarrow \text{SHAKE}(\text{DID} \parallel \text{state.mdk.bdk})$
 - 4: $\text{CTOK} \leftarrow \text{TRC} \parallel \text{FN} \parallel \text{DID}$ using $\text{TRC} = \lfloor \text{KTC} / \text{HKDS_CACHE_SIZE} \rfloor$
 - 5: $\text{TOK} \leftarrow \text{SHAKE}(\text{CTOK} \parallel \text{state.mdk.stk})$
 - 6: $o \leftarrow j \cdot \text{HKDS_MESSAGE_SIZE}$
 - 7: $r \leftarrow \lceil (o + \text{tkeylen}) / \text{HKDS_PRF_RATE} \rceil$
 - 8: squeeze only r SHAKE rate blocks from $\text{SHAKE}(\text{TOK} \parallel \text{EDK})$ into a temporary buffer
 - 9: copy the requested window beginning at offset o into the output key buffer
-

Algorithm 7 Server token encryption

Require: Server state `state`

- 1: $\text{DID} \leftarrow \text{state.ksn}[0..\text{HKDS_DID_SIZE}-1]$
 - 2: $\text{EDK} \leftarrow \text{SHAKE}(\text{DID} \parallel \text{state.mdk.bdk})$
 - 3: $\text{CTOK} \leftarrow \text{TRC} \parallel \text{FN} \parallel \text{DID}$
 - 4: $\text{TOK} \leftarrow \text{SHAKE}(\text{CTOK} \parallel \text{state.mdk.stk})$
 - 5: $\text{Kpad} \leftarrow \text{SHAKE}(\text{CTOK} \parallel \text{EDK})$
 - 6: $\text{ETOK} \leftarrow \text{TOK} \oplus \text{Kpad}$
 - 7: $\text{TMS} \leftarrow \text{KSN} \parallel \text{MACName}$
 - 8: $\text{TAG}_t \leftarrow \text{KMAC}_{\text{EDK}}(\text{ETOK}; \text{custom} = \text{TMS})$ truncated to 16 bytes
 - 9: **return** $\text{ETOK} \parallel \text{TAG}_t$
-

Algorithm 8 Server authenticated verification and decryption

Require: Server state `state`, received $C \parallel T$, associated data A

- 1: reconstruct a 32 byte transaction-key window
 - 2: parse the window as $K_{\text{enc}} \parallel K_{\text{mac}}$
 - 3: $T^* \leftarrow \text{KMAC}_{K_{\text{mac}}}(C; \text{custom} = A)$ truncated to 16 bytes
 - 4: **if** $T^* \neq T$ **then**
 - 5: **return** failure
 - 6: **end if**
 - 7: $M \leftarrow C \oplus K_{\text{enc}}$
 - 8: **return** M
-

3 Preliminaries and Assumptions

3.1 Notation

We write $x \parallel y$ for byte-string concatenation and $x \oplus y$ for bitwise exclusive OR on equal-length strings. The selected HKDS root-key profile is denoted by

$$\lambda \in \{128, 256, 512\}.$$

The fixed message block, transaction-key block, token tag, and message tag width is denoted by

$$\tau = 128.$$

Thus λ determines the width of BDK, STK, EDK, and TOK, while τ determines the online message-authentication and transaction-key block bound in the implementation analyzed here.

Table 1: HKDS parameter and identifier sizes

Parameter	Size
Device identity block DID	96 bits
Key serial number KSN = DID KTC	128 bits
Base derivation key BDK	128/256/512 bits
Secret token key STK	128/256/512 bits
Embedded device key EDK	128/256/512 bits
Token value TOK and encrypted-token body ETOK	128/256/512 bits
Token response payload ETOK TAG _t	256/384/640 bits
Token and message authentication tag	128 bits fixed
CTOK customization string	184 bits
TMS customization string	184 bits
Message block and transaction-key block	128 bits
Transaction cache entries	42/34/18
SHAKE rate	168/136/72 bytes

3.2 Primitive Assumptions

HKDS uses SHAKE as an extendable-output pseudorandom function on fixed absorbed inputs and KMAC as a keyed pseudorandom function for authentication. The analysis assumes the following.

- A1.** SHAKE is a PRF on the HKDS absorbed input domains at the selected profile.
- A2.** KMAC is a PRF under uniformly random keys and fixed customization strings.
- A3.** BDK and STK are independently and uniformly generated.
- A4.** KID and DID values are unique in the deployment domain where uniqueness is required by the protocol.
- A5.** KTC values do not repeat under the same device identity and master-key context.
- A6.** Consumed transaction-key entries and expired token material are erased before adversarial corruption in the forward-secrecy experiment.

Physical side channels, fault injection, power analysis, electromagnetic leakage, compiler removal of intended memory erasure, and denial-of-service behavior are outside the cryptographic proof model. They are addressed as implementation requirements in Section 8.

4 Security Model

4.1 Adversary

The adversary controls the communication channel. It may observe, drop, replay, modify, and inject token packets and message packets. It may request honest encryptions, attempt server verification of chosen ciphertexts, observe token responses, and corrupt clients subject to the freshness restrictions

of each experiment. Unless a theorem explicitly states otherwise, the server root values BDK and STK remain secret.

4.2 Oracles

The formal experiments use the following oracles.

NewEpoch(C). For client C , this oracle forms the current KSN, computes CTOK and TMS, derives TOK and Kpad, forms ETOK, computes the token tag, and returns (ETOK, TAG _{t} , KSN).

Encrypt(C, M, A). This oracle consumes two transaction-key entries for client C , returns ($Ctxt, T$, KSN) where $Ctxt = M \oplus K_{\text{enc}}$ and $T = \text{KMAC}_{K_{\text{mac}}}(Ctxt; A)$ truncated to 16 bytes, and advances the KTC twice.

VerifyDecrypt($C, Ctxt, T, A$, KSN). This oracle reconstructs the relevant key window from KSN, verifies T , and returns either reject or the plaintext. Authenticity games count success when the adversary causes acceptance for a pair not output by Encrypt.

CorruptClient(C). This oracle returns the readable client state, including EDK, KSN, cache-empty state, and any unconsumed non-erased cache entries. It does not return erased entries or server-only roots.

Test(C, M_0, M_1, A). This oracle chooses a hidden bit b , computes an authenticated encryption of M_b , and returns the challenge transcript. It is unavailable for a corrupted challenge client.

4.3 Security Notions

Message confidentiality is defined by indistinguishability of Test encryptions under chosen-plaintext attack. Message authenticity is defined by unforgeability of authenticated ciphertexts under chosen-message attack. Token authenticity is defined by the inability to produce a new token response that the client accepts. Forward secrecy is defined by indistinguishability of past transaction keys after erasure and subsequent client corruption. Post-compromise security is defined by indistinguishability of keys in a fresh token epoch after a prior client compromise, assuming STK remains secret.

5 Structural Properties

5.1 Domain Separation

HKDS uses SHAKE in four principal domains:

$$X_1 = \text{DID} \parallel \text{BDK}, \quad X_2 = \text{CTOK} \parallel \text{STK}, \quad X_3 = \text{CTOK} \parallel \text{EDK}, \quad X_4 = \text{TOK} \parallel \text{EDK}.$$

The byte lengths are respectively

$$12 + \lambda/8, \quad 23 + \lambda/8, \quad 23 + \lambda/8, \quad 2\lambda/8.$$

All pairs except X_2 and X_3 are separated by fixed input length at each supported profile. The remaining pair has the same length and public prefix, but independent secret suffixes: STK is sampled uniformly, while EDK is derived as $\text{SHAKE}(\text{DID} \parallel \text{BDK})$ and is computationally indistinguishable from uniform under the SHAKE PRF assumption.

Lemma 5.1 (Computational domain separation). *Under Assumptions A1 through A3, no efficient adversary can obtain a non-negligible advantage by confusing two HKDS SHAKE domains or two HKDS KMAC domains.*

Proof. For SHAKE, all domain pairs except X_2 and X_3 are distinguished by length. For X_2 and X_3 , fix a public CTOK. If an adversary distinguishes or relates outputs on $\text{CTOK} \parallel \text{STK}$ and $\text{CTOK} \parallel \text{EDK}$ with non-negligible advantage, then either it distinguishes EDK from a uniform string or it distinguishes SHAKE from a random function on two independent equal-length inputs. The first case contradicts SHAKE PRF security on $\text{DID} \parallel \text{BDK}$; the second contradicts SHAKE PRF security on the token and pad domains.

For KMAC, token authentication uses key EDK and customization string TMS, while message authentication uses a transaction-key block and associated data. The keys are outputs of separated SHAKE domains. If an adversary transfers a tag or distinguishing advantage between these domains, a KMAC PRF distinguisher can embed its challenge oracle in the attacked domain and simulate all other domains with independently sampled keys. Hence cross-domain confusion implies a PRF break against SHAKE or KMAC. $\square$

5.2 Deterministic Recoverability

Lemma 5.2 (Deterministic recoverability). *For fixed $(\text{BDK}, \text{STK}, \text{KID})$, DID , and KSN , the tuple $(\text{EDK}, \text{TOK}, \text{TKC})$ is uniquely determined. An honest client holding the corresponding EDK and token and an honest server holding the MDK compute the same transaction-key entries for the same KTC.*

Proof. The value $\text{EDK} = \text{SHAKE}(\text{DID} \parallel \text{BDK})$ is deterministic. The KSN determines KTC and therefore TRC. CTOK is the fixed concatenation $\text{TRC} \parallel \text{FN} \parallel \text{DID}$, and $\text{TOK} = \text{SHAKE}(\text{CTOK} \parallel \text{STK})$ is deterministic. Finally, $\text{TKC} = \text{SHAKE}(\text{TOK} \parallel \text{EDK})$ is deterministic and is partitioned into fixed 16 byte entries. Therefore both parties compute the same key window when given the same KSN and master-key context. $\square$

6 Security Results

6.1 Message Confidentiality

Theorem 6.1 (IND-CPA security). *Let $\mathcal{A}$ make at most q_{enc} encryption or test queries against authenticated HKDS without corrupting the challenge client. There exists a SHAKE PRF adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{HKDS}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{\text{SHAKE}}^{\text{PRF}}(\mathcal{B}) + \frac{q_{\text{enc}}^2}{2^{\tau+1}},$$

where $\tau = 128$.

Proof. Game G_0 is the real experiment. In G_1 , all SHAKE outputs used to derive transaction-key cache material for fresh token epochs are replaced by consistent random-function outputs. Any

difference between G_0 and G_1 yields a SHAKE PRF distinguisher. In G_1 , each challenge encryption key is a fresh uniform 128-bit block used once. Therefore $C = M_b \oplus K_{\text{enc}}$ is uniformly distributed over 128-bit strings independent of b . The accompanying tag is computed under an independent transaction-key block, so it also does not reveal b except through collision events among sampled blocks or transcripts. The birthday bound over at most q_{enc} challenge-related uses gives $q_{\text{enc}}^2/2^{\tau+1}$. In the final game the adversary's view is independent of b , establishing the bound. $\square$

6.2 Message Authenticity

Theorem 6.2 (UF-CMA security). *Let $\mathcal{A}$ make at most q_{mac} chosen-message encryption queries and then output a new authenticated ciphertext accepted by the server. There exist SHAKE and KMAC PRF adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{HKDS}}^{\text{UF-CMA}}(\mathcal{A}) \leq \text{Adv}_{\text{SHAKE}}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{\text{KMAC}}^{\text{PRF}}(\mathcal{B}_2) + \frac{q_{\text{mac}}}{2^\tau}.$$

Proof. First replace SHAKE-derived MAC keys with independent uniform 128-bit keys. The replacement cost is bounded by SHAKE PRF security. With independent MAC keys, each valid tag is a KMAC output under a fresh key. A KMAC PRF reduction embeds its challenge oracle in the forged key instance and simulates all other instances with independent keys. If KMAC is replaced by a random function, a new valid tag can only be guessed, contributing at most $q_{\text{mac}}/2^\tau$. Combining the SHAKE replacement, KMAC PRF, and generic guessing terms proves the theorem. $\square$

6.3 Token Authenticity

Theorem 6.3 (Token authenticity). *Let $\mathcal{A}$ observe at most q_{tok} valid token responses and then output a new pair $(\text{ETOK}^*, \text{TAG}_t^*)$ accepted by a client. There exists a KMAC PRF adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{HKDS}}^{\text{TokAuth}}(\mathcal{A}) \leq \text{Adv}_{\text{KMAC}}^{\text{PRF}}(\mathcal{B}) + \frac{q_{\text{tok}}}{2^\tau}.$$

Proof. A client accepts only if the fixed 16 byte KMAC tag verifies under EDK with customization string TMS. Under the SHAKE assumption on device-key derivation, EDK is computationally indistinguishable from a uniform KMAC key to an adversary that does not corrupt the client. A KMAC PRF adversary can answer token-tag queries using its oracle and forward a successful new token forgery as a MAC forgery. In the random-function case a new tag succeeds with probability at most $2^{-\tau}$ per attempt, giving the stated bound. $\square$

6.4 Forward Secrecy Under Erasure

Definition 6.4 (Erasure model). *At the end of an epoch, all expired token material, token pads, local transaction-key copies, and consumed transaction-key cache entries are overwritten. A corruption after erasure returns only the remaining readable state.*

Theorem 6.5 (Forward secrecy). *If a client is corrupted only after the erasure step for epoch i , then any adversary's advantage in distinguishing a consumed key from epoch i from random is bounded by a SHAKE PRF advantage, subject to the erasure model.*

Proof. After erasure, the client state no longer contains the target key, the token that generated it, or the cache material from which it was copied. A simulator embeds its SHAKE challenge in the

derivation $\text{SHAKE}(\text{TOK}_i \parallel \text{EDK})$ for the challenge epoch, while simulating other epochs and oracles directly. If the adversary distinguishes the erased key from random after corruption, the simulator distinguishes real SHAKE output from random output in the challenge cache domain. $\square$

6.5 Post-Compromise Security

Theorem 6.6 (Post-compromise security). *Suppose an adversary corrupts a client at the end of epoch i and learns EDK , TOK_i , and unconsumed cache entries. If STK remains secret and the client obtains a fresh token for epoch $i + 1$, then keys derived in epoch $i + 1$ are indistinguishable from random up to SHAKE PRF advantage.*

Proof. The adversary knows EDK but not STK . The next token is $\text{TOK}_{i+1} = \text{SHAKE}(\text{CTOK}_{i+1} \parallel \text{STK})$ with a new token-request counter. Under SHAKE PRF security and secrecy of STK , TOK_{i+1} is indistinguishable from a fresh random value to the adversary. The next cache is $\text{SHAKE}(\text{TOK}_{i+1} \parallel \text{EDK})$. Since the first component is fresh and pseudorandom, a distinguisher for the resulting cache gives a SHAKE PRF distinguisher for the cache domain. $\square$

6.6 Multi-Device Separation

Theorem 6.7 (Cross-device independence). *For distinct device identity blocks $\text{DID}_a \neq \text{DID}_b$ under the same BDK , compromise of the readable state of device a does not reveal the EDK , token pad, token, cache, or transaction keys of device b , except with SHAKE PRF advantage and the usual multi-instance loss.*

Proof. The embedded keys are $\text{EDK}_a = \text{SHAKE}(\text{DID}_a \parallel \text{BDK})$ and $\text{EDK}_b = \text{SHAKE}(\text{DID}_b \parallel \text{BDK})$. Distinct fixed prefixes define distinct PRF inputs. Replacing SHAKE by a random function gives independent random outputs for distinct inputs. Therefore learning EDK_a and device a cache material does not reveal EDK_b or any value that depends on it. A union bound accounts for multiple attacked devices. $\square$

6.7 Partial Server-Root Compromise

If BDK is exposed while STK remains secret, an adversary can compute EDK for observed device identities. It can decrypt observed encrypted token bodies by deriving $\text{Kpad} = \text{SHAKE}(\text{CTOK} \parallel \text{EDK})$ and can therefore recover token values for observed token responses. Confidentiality of sessions observable to the adversary is lost for affected devices.

If STK is exposed while BDK remains secret, an adversary can compute token values $\text{TOK} = \text{SHAKE}(\text{CTOK} \parallel \text{STK})$ for observed CTOK values, but cannot derive EDK , token pads, or transaction-key caches without BDK or an exposed device EDK . Thus neither root alone provides the complete set of inputs for all HKDS derivations, although exposure of either root is a serious operational event requiring rotation and incident response.

7 Concrete Bounds

Let $\varepsilon_{\text{SHAKE}}$ denote the maximum SHAKE PRF advantage for comparable resources, and let $\varepsilon_{\text{KMAC}}$ denote the corresponding KMAC PRF advantage. Let $\tau = 128$. The preceding results yield:

$$\begin{aligned}
\text{Adv}_{\text{HKDS}}^{\text{IND-CPA}}(\mathcal{A}) &\leq \varepsilon_{\text{SHAKE}} + \frac{q_{\text{enc}}^2}{2^{\tau+1}}, \\
\text{Adv}_{\text{HKDS}}^{\text{UF-CMA}}(\mathcal{A}) &\leq \varepsilon_{\text{SHAKE}} + \varepsilon_{\text{KMAC}} + \frac{q_{\text{mac}}}{2^{\tau}}, \\
\text{Adv}_{\text{HKDS}}^{\text{TokAuth}}(\mathcal{A}) &\leq \varepsilon_{\text{KMAC}} + \frac{q_{\text{tok}}}{2^{\tau}}, \\
\text{Adv}_{\text{HKDS}}^{\text{FS}}(\mathcal{A}) &\leq \varepsilon_{\text{SHAKE}}, \\
\text{Adv}_{\text{HKDS}}^{\text{PCS}}(\mathcal{A}) &\leq \varepsilon_{\text{SHAKE}}.
\end{aligned}$$

Table 2: Concrete security bounds for authenticated HKDS

Property	Resources	Upper bound
IND-CPA confidentiality	q_{enc} encrypt/test queries	$\varepsilon_{\text{SHAKE}} + q_{\text{enc}}^2/2^{\tau+1}$
UF-CMA authenticity	q_{mac} MAC queries	$\varepsilon_{\text{SHAKE}} + \varepsilon_{\text{KMAC}} + q_{\text{mac}}/2^{\tau}$
Token authenticity	q_{tok} token attempts	$\varepsilon_{\text{KMAC}} + q_{\text{tok}}/2^{\tau}$
Forward secrecy	erasure-respecting corruption	$\varepsilon_{\text{SHAKE}}$
Post-compromise security	fresh token after corruption	$\varepsilon_{\text{SHAKE}}$

The online tag-guessing and transaction-key block terms are governed by $\tau = 128$ because the implementation fixes message blocks, transaction-key entries, and tags at 16 bytes. The 256-bit and 512-bit profiles increase the security margin of BDK, STK, EDK, token generation, and token encryption, but do not by themselves enlarge the per-message tag or transaction-key width. A profile that requires higher online MAC security must also enlarge the tag and transaction-key-block parameters in a compatible specification and implementation.

8 Implementation and System Considerations

8.1 Erasure and Memory Hygiene

The formal erasure model is stronger than ordinary stack deallocation. Implementations should explicitly erase token pads, decrypted token buffers after cache generation, local encryption and MAC key copies, temporary SHAKE output buffers, and consumed cache entries. Cache-entry erasure during extraction is essential for forward secrecy. Caller-owned buffers must be included in the erasure policy, because token decryption writes the recovered token into an output buffer used by cache generation.

8.2 Counter Discipline and Replay

KTC is a 32-bit field. The security model assumes that a KTC value is never reused under the same DID and master-key context. A deployment must retire, renew, or re-provision the device context before KTC wraps to zero. Reuse of KTC can repeat CTOK and the cache index, invalidating the freshness assumptions used in the confidentiality, authenticity, and replay-resistance arguments.

The cryptographic core reconstructs keys from the received KSN; it does not by itself maintain a persistent anti-replay database. A deployed transaction system must reject stale KSN values,

enforce monotonicity per device, and treat repeated verification failures as synchronization faults or compromise indicators.

8.3 Authenticated and Unauthenticated Modes

The authenticated KMAC mode is the mode analyzed for integrity. The unauthenticated XOR-only mode provides only confidentiality under the single-use key assumption and provides no message integrity or ciphertext replay protection. Payment and high-assurance deployments should use authenticated mode unless another authenticated channel or higher-layer integrity mechanism is formally specified.

8.4 Parallel APIs

The x8 and x64 server APIs compute scalar HKDS derivations independently across lanes. Their security is inherited from the scalar construction when per-lane inputs are distinct as required by the protocol, vector lanes do not alias memory, and the vectorized Keccak/KMAC implementation preserves constant-time behavior and memory isolation. A lane-level implementation defect is outside the formal scalar proof and must be evaluated by code review and testing.

8.5 Side Channels and Reduced Rounds

The proofs assume ideal constant-time implementations of SHAKE and KMAC. Timing, power, electromagnetic leakage, cache behavior, and fault responses are outside the model. Full-round Keccak should be used for high-assurance deployments. Reduced-round configurations are engineering variants whose security margin must be assessed separately and should not be treated as covered by the full-round proof.

9 Conclusion

This paper formalized HKDS as a deterministic symmetric key distribution protocol with token-based key epochs, per-device embedded keys, and one-time transaction-key cache entries. The analysis established confidentiality, message authenticity, token authenticity, forward secrecy under erasure, post-compromise security, multi-device separation, and partial server-root compromise behavior under standard PRF assumptions for SHAKE and KMAC.

The principal conditions for the security results are strict KTC uniqueness, correct construction of CTOK as $\text{TRC} \parallel \text{FN} \parallel \text{DID}$, fixed interpretation of DID and KSN according to the implementation, use of authenticated KMAC mode for integrity, and explicit erasure of expired key material. The concrete bounds distinguish the selected 128/256/512-bit root-key profile from the fixed 128-bit online tag and transaction-key widths. Under these conditions, HKDS provides a compact, deterministic, server-reconstructible symmetric key establishment mechanism suitable for high-volume transaction environments.

References

- [1] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. *The Keccak Reference*. Submission to the NIST SHA-3 Competition, 2011. <https://keccak.team/files/Keccak-reference-3.0.pdf>
- [2] National Institute of Standards and Technology. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. FIPS Publication 202, 2015. <https://doi.org/10.6028/NIST.FIPS.202>
- [3] National Institute of Standards and Technology. *SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*. NIST Special Publication 800-185, 2016. <https://doi.org/10.6028/NIST.SP.800-185>
- [4] Accredited Standards Committee X9. *Derived Unique Key Per Transaction (DUKPT)*. ANSI X9.24-3, 2016. <https://www.x9.org/standards/>
- [5] V. Shoup. Sequences of Games: A Tool for Taming Complexity in Security Proofs. Cryptology ePrint Archive, Report 2004/332, 2004. <https://eprint.iacr.org/2004/332>
- [6] M. Bellare and P. Rogaway. Entity Authentication and Key Distribution. In *Advances in Cryptology - CRYPTO 1993*. https://doi.org/10.1007/3-540-48329-2_24
- [7] M. Bellare, R. Canetti, and H. Krawczyk. Keying Hash Functions for Message Authentication. In *Advances in Cryptology - CRYPTO 1996*. https://doi.org/10.1007/3-540-68697-5_1
- [8] H. Krawczyk. Cryptographic Extraction and Key Derivation: The HKDF Scheme. In *CRYPTO 2010*. https://doi.org/10.1007/978-3-642-14623-7_33
- [9] J. G. Underhill. *Hierarchal Symmetric Key Distribution System - HKDS*. HKDS specification manuscript.
- [10] J. G. Underhill. *HKDS Reference C Implementation*. Source files `hkds_client.c`, `hkds_server.c`, and `hkds_config.h`.